



ALL



1

2

3

4

## 2. Efficient Mining Algorithm

Miners are trying to mine a cryptocurrency which is based on the amount of coal they have and the quantity of energy that they can consume from the coal. They are provided with two arrays :

- one indicating the amount of coal they have as separate reservoirs, and
- the energy that can be required for mining.

The cryptocurrency can be mined if the following constraint is satisfied :

**minimum of the energy requirement array  $\geq$  maximum of the coal amount array**

**However they can add or subtract 1 to any index of any array, but doing any operation will result in a unit cost.**

The miner want themselves to be as efficient as possible, your task is to help miner solve this problem with minimum cost that can be required so that a given cryptocurrency can be mined.

### Input:

The first line contains two space-separated integers  $n, m$  denoting the size of the arrays.

The second line will contain  $n$  space-separated integers representing content of the array **coal** [ $c$ ] (*size =  $n$* ) (*indicating the amount of coal as an array*).

The third line will contain  $m$  space-separated integers representing content of the array **energy** [ $e$ ] (*size =  $m$* ) (*amount of energy required as an array*).

### Constraints:

$$1 \leq n, m \leq 10^4$$

$$1 \leq c_i \leq 10^{12}$$

$$1 \leq e_i \leq 10^{12}$$

### Sample input 1:

```
3 3
2 4 3
1 2 3
```

### Output 1:

```
4
```

### Explanation 1:

Consider 1 based indexing remove 1 coal from  $c_2$  coal array becomes [2, 3, 3]  $\max = 3$ , add 1 energy to  $e_1$ , energy array becomes [2, 2, 3], add 1 energy again to  $e_1$ , energy array is [3, 2, 3], add 1 energy to  $e_2$ , the energy array becomes [3, 3, 3],  $\min = 3$

Thus by 4 operations, minimum of energy requirement array  $\geq$  maximum of coal amount array.

Though there are other ways also possible to make this condition true but 4 will be our most optimal answer.

### Sample input 2:

```
3 3
2 4 3
8 5 9
```

### Output 2:

```
0
```



Autocomplete Ready ⓘ

```
3 #include <list>
4 #include <cmath>
5 #include <ctime>
6 #include <deque>
7 #include <queue>
8 #include <stack>
9 #include <string>
10 #include <bitset>
11 #include <cstdio>
12 #include <limits>
13 #include <vector>
14 #include <climits>
15 #include <cstring>
16 #include <cstdlib>
17 #include <fstream>
18 #include <numeric>
19 #include <sstream>
20 #include <iostream>
21 #include <algorithm>
```